

Pipeline example

Introduction

This file shows how to utilize the pipeline built here to generate an ERC network for some species of interest. In this case, we will be focusing on *D. melanogaster* and utilizing all annotated Dipteran genomes available on ncbi, alongside *C. felis* as an outgroup. We assume that you are using some form of Unix, but the pipeline has been tested on Ubuntu running in WSL2. It *should* work in both Windows and on MacOS, assuming all necessary software is installed and visible to your CLI.

Initializing the software, installing all dependencies.

Before we run any analyses, we will require several tools. 1) Julia v 1.12+

Once Julia is installed, run the below script:

```
using Pkg
Pkg.add("DrWatson")
Pkg.activate("CoEvolution")
Pkg.instantiate()
```

This might run for a while, as it will compile any necessary packages for later scripts. While that is running, install the following on your system.

- 2) [Mafft](#)
- 3) [Diamond](#)
- 4) [iqTree2](#)

If you would like to use this pipeline to download and prep data, make sure you have installed:

- 5) [datasets-cli](#)
- 6) [ncbi e-utils](#)

7) seq-kit

Hopefully Julia is up and running at this point, so we can begin on the main steps of the analysis.

Defining major parameters

The below codeblock is where you can define the major parameters for the pipeline (taxon, focal species, outgroup, directory). The nature of this pipeline means it might be a good idea to just have these in the `Project.toml` file, but I know people will be more afraid of editing a file they can't see versus running code-blocks in a somewhat familiar interface.

```
using DrWatson
@quickactivate

#Set these values for your own analysis. Taxon can be given as plain-text, but use ncbi taxon
taxon = "Diptera"
data_dir = datadir("dipterans/") #The default place for all of the files, but give the absolute path
#Specify the focal species for the analysis.
focal= "D_melanogaster"
#Specify an outgroup - necessary for ASTRAL-IV to adequately make a species tree.
outgroup_ncbi_id = "Ctenocephalides felis"
outgroup = "C_felis"
include(srcdir("Stats_utilities.jl"))
include(srcdir("Phylo_utilities.jl"))
```

Prep data

You can prepare the data in one of two ways. First, you can define the taxon of interest and run the code block below to download data using our internal script. This will poll the NCBI `datasets` database to find any annotated, chromosome level assemblies within the taxon specified, download their protein sequences and gff files and place them in the data directory in the `seqs` folder. Alternatively, if you already have full amino acid sequences for a set of species of interest, simply create a `seqs` folder in `data_dir` and you should be able to skip to the next step.

```
#SKIP THIS IF YOU ARE USING YOUR OWN DATA!
include(srcdir("Finding_orthologs.jl"))
isdir(data_dir) || mkdir(data_dir)
```

```
download_and_prep_sequences(taxon)
download_and_prep_sequences(outgroup_ncbi_id;level="scaffold")
```

Finding orthologs for focal species.

We first identify orthologous sequences for all genes within the focal species. This is done in the following code block, most functions are defined in the `src\Finding_orthologs.jl` script. We load in a bunch of helper functions from the `ERC_functions.jl` source file as well.

```
include(scrdir("Finding_orthologs.jl"))
include(scrdir("ERC_functions.jl"))
```

The main bulk of computational time is in the below block. Detecting reciprocal best hit orthologs, creating multi-sequence alignments and then generating a tree for each alignment. Run `julia` with as many threads as you can spare, as each step is parallelized within.

```
#Now run the below, and go off to grab some tea, or take a break for the day.
species = replace.(filter(endswith(".faa"),readdir(data_dir*"seqs/")),r".faa"=>"")
all_orthologs = find_orths(focal,species,data_dir)
#If the above gets interrupted - delete any .tsv files of size 0 in your `data_dir\blast` dir
using Arrow

Arrow.write(data_dir*focal*"_orthologs.arrow",all_orthologs)

#And if things break later, you can always load in the all_orthologs dataframe with `all_orthologs`

#The next step takes a while, since parallelization is doable but can backfire (mafft can take a while)
mkdir(data_dir*focal*"/") || mkdir(data_dir*focal*"/")
build_orth_files(all_orthologs,focal,data_dir)
```

Tree Generation

Next we generate trees for each gene. Earlier versions we constrained the tree topology (and you'll find functions in `src/Phylo_utilities.jl` that are capable of doing that), but I realized more and more that this was likely leading to some false negatives in the signal for co-evolution. Currently the code is tested in IQTree2, but IQTree3 should work, and requires a simple change to the `run_iqtree` function in `src/Phylo_utilities.jl`. The end of the script generates a consensus species tree using ASTRAL. This is the only step we really need the outgroup for, as it allows for better estimates of branch lengths.

```

using FLoops
using ProgressMeter

isdir(data_dir*focal*"/trees/") || mkdir(data_dir*focal*"/trees/")

aligned_genes = filter(endswith(".aligned.fasta"),readdir(data_dir*focal*"/aligned/",join=true))

p=Progress(length(aligned_genes),desc="Running IQTree2")
@floop ThreadedEx() for seq in aligned_genes
    try
        run_iqtree(seq;path=data_dir*focal*"/trees/",model="Q.insect,NQ.insect,LG,JTT")
    catch
    end
    next!(p)
end

finished_trees = filter(endswith(".treefile"),readdir(data_dir*focal*"/trees/",join=true))
#Create consensus with ASTRAL-IV
run_astral(finished_trees,outgroup)
#Reroot in R to focal
tree_loc =data_dir*focal*"trees/astral_consensus_rerooted.newick"
cmd=`sed -i "s/)[0-1]\\.[0-9]*:/:/g" $tree_loc`
run(cmd)

```

Calculating ERC

If you are using your own trees, start here!

You will need to specify the species tree and the list of gene trees. If you have followed the pipeline so far, they are located in the `data/taxon/focal/trees` folder.

```

#Change if using some other species tree.
species_tree = read_tree(data_dir*focal*"/trees/astral_consensus_rerooted.newick")
tree_files = filter(endswith(".treefile"),readdir(data_dir*focal*"/trees/",join=true))

#This takes a while, but saves a lot of reading time in the ERC loop. It does increase memory
trees = [read_tree(t) for t in tree_files]
ERC = runERC_collection(trees,species_tree;cutoff=5,min_shared=4)

#Drop cases with missing data
ERC = ERC[findall(completecases(ERC)),:]
disallowmissing!(ERC)

```

```

genes = replace.(filter(endswith(".treefile"),readdir(data_dir*focal*"/trees/"),join=false)),

ERC.i = genes[Int.(ERC.i)]
ERC.j = genes[Int.(ERC.j)]
ERC.gid = [join(sort([x.i,x.j]),"_") for x in eachrow(ERC)]

using Arrow
Arrow.write(data_dir*focal*"_ERC.arrow",ERC)
ERC = DataFrame(Arrow.Table(data_dir*focal*"_ERC.arrow"))

```

Gene Table Annotation and comparisons

We next build a table that includes all of the gene information. This is based off of the .gff file for your reference of interest. If you don't have an annotation, this *is* skippable, but helps look for any potential issues with increased signal for genes next to each other on chromosomes, etc.

One element you will need to define manually is `chr_dict`. The `chr_dict` is just a quick conversion from NCBI's chr ids to something human readable - you'll have to make your own for your system. Note, we drop anything that's on an unmapped scaffold in the process, in our case 6 genes. If you have hundreds of chromosomes/scaffolds, this step can be skipped, or you can use some other approach to rename your regions.

```

include(sourcedir("Annotation_functions.jl"))

genes = replace.(filter(endswith(".treefile"),readdir(data_dir*"trees/"),join=false)),r".tree

gene_table = DataFrame(gene=genes,
    stable_id=missings(String,length(genes)),
    name=missings(String,length(genes)),
    flybase=missings(String,length(genes)),
    chr=missings(String,length(genes)),
    pos_1=missings(Int64,length(genes)),
    pos_2=missings(Int64,length(genes)),
    description=missings(String,length(genes)),
    num_rbh=missings(Int64,length(genes)))

#Can't query all 12k at once, split into chunks of 200
part_size=200
parts = collect(Iterators.partition(genes,part_size))
for gs in 1:length(parts)
    gene_names = copy(parts[gs])

```

```

    subtable = find_name(gene_names,"dmelanogaster_gene_ensembl")
    rows = indexin(subtable.gene,genes)
    gene_table[rows,2:8] = subtable[:,2:8]
    gene_table.num_rbh[(1:length(gene_names)).+part_size*(gs-1)] = num_rbh.(gene_names)
end

#For genes with no Ensembl annotation, we look in the gff file to at least get location data

using CSV

gff_table = CSV.read(data_dir*focal*"_genes.gff",DataFrame,header=false,skipto=9)
gff_table = gff_table[completecases(gff_table),:]

chr_dict = Dict("NC_004354.4"=>"X","NT_033779.5"=>"2L","NT_033778.4"=>"2R","NT_037436.4"=>"3R")

gff_table.Column1 = [get(chr_dict,c,"NA") for c in gff_table.Column1]
to_do_genes = findall(ismissing.(gene_table.chr))
for g in to_do_genes
    subtable = gff_table[findall(occursin.(genes[g],gff_table.Column9)),:]
    if size(subtable)[1] > 0
        gene_table.chr[g] = subtable.Column1[1]
        gene_table.pos_1[g] = minimum(subtable.Column4)
        gene_table.pos_2[g] = maximum(subtable.Column5)
    end
end
end

CSV.write(data_dir*focal*"_gene_table.csv",gene_table)

```

ERC network analysis

Finding significant edges

```

#BY correct p-values
using MultipleTesting
ERC.pval_BY = adjust(ERC.pval,BenjaminiYekutieli())

include(sourcedir("ERC_simulations.jl"))
species_tree = read_tree(data_dir*focal*"/trees/astral_consensus_rerooted.newick")

#Positive null - trees that _have_ evolved along the exact same rates, so should show correlation

```

```

trees_null = simulate_tree_set(species_tree,10,1000,40,73,focal)
ERC_sims = runERC_collection(trees_null,species_tree;cutoff=5)
ERC_sims = ERC_sims[completecases(ERC_sims),:]
disallowmissing!(ERC_sims)
sims_cdf = ecdf(ERC_sims.fERC)
ERC.sims = sims_cdf.(ERC.fERC)

#Negative null - correlations between i.i.d. branch lengths drawn from exponential proportion
ERC_null = calculate_ERC_exp_null(species_tree,1000)
disallowmissing!(ERC_null)
ERC_null.fERC = fisher_trans.(ERC_null.r,length(get_branch_lengths(species_tree)))
null_cdf = ecdf(ERC_null.fERC)
ERC.null = null_cdf.(ERC.fERC)

#Take only edges outside of expectation from positive sim. No good a priori cut-off to choose
gt_sims = findall(ERC.sims .>0.95)

#Run BH correction on resulting interactions to retain only edges with FDR 0.001 (combinatorial)
#Now, edges that are Bonferonni significant _and_ beyond dynamic cut-off are retained further

CSV.write(data_dir*focal*"_sig_ints.csv",sig_ERC)

```

Broad analysis of scores

We start by looking at the broad set of ERC scores. How do they compare to some potential theoretical nulls, and do they map to any interesting gene features?

```

gene_table = CSV.read(data_dir*focal*"_gene_table.csv",DataFrame)
using Arrow
ERC = DataFrame(Arrow.Table(data_dir*focal*"_ERC.arrow"))

id_i = indexin(ERC.i,gene_table.gene)
id_j = indexin(ERC.j,gene_table.gene)

ERC.chr_i = gene_table.chr[id_i]
ERC.chr_j = gene_table.chr[id_j]

ERC.pos_i_1 = gene_table.pos_1[id_i]
ERC.pos_i_2 = gene_table.pos_2[id_i]

```

```

ERC.pos_j_1 = gene_table.pos_1[id_j]
ERC.pos_j_2 = gene_table.pos_2[id_j]

ERC.dist = missings(Int64,length(ERC.i))
intra = findall(ERC.chr_i .== ERC.chr_j)
ERC.same_chr .= "different"
ERC.same_chr[intra] .= "same"
ERC.dist[intra] = [range_dist([x.pos_i_1,x.pos_i_2],[x.pos_j_1,x.pos_j_2]) for x in eachrow(ERC)]

Arrow.write(data_dir*focal*"_ERC_annotated.arrow",ERC)

```

Next, some summaries for the gene_table:

```

#Annotate the gene table.sig_
function indexInERC(gene); findall(ERC.i .== gene || ERC.j .== gene);end
idx = map(indexInERC,gene_table.gene)
gene_table.sig_ints = [length(union(findall(sig_ERC.i==x),findall(sig_ERC.j==x))) for x in gene_table.gene]
gene_table.mean_r = [mean(ERC.r[idx[i]]) for i in 1:length(idx)]
gene_table.mean_fERC = [mean(ERC.fERC[idx[i]]) for i in 1:length(idx)]
gene_table.mean_shared = [mean(ERC.n_edges[idx[i]]) for i in 1:length(idx)]
CSV.write(data_dir*focal*"_annotated_gene_table.csv",gene_table)

```

Now we generate some basic summary plots for the data:

```

ERC = DataFrame(Arrow.Table(data_dir*focal*"_ERC_annotated.arrow"))

using CairoMakie, LaTeXStrings
include(sourcedir("Plot_utilities.jl"))
isdir(data_dir*"plots") || mkdir(data_dir*"plots")
plot_prefix=data_dir*"plots/"*focal*"_ "

#Panel A: Histograms of fERC for three distributions
fig1 = Figure(size=(800,800))
ax1 = Axis(fig1[1,1],
    xlabel="fERC",
    ylabel="Frequency",
    xgridvisible=false,
    ygridvisible=false)

hist!(ax1,ERC_sims.fERC,bins=100,
    label="Positive Simulations",normalization=:pdf,

```



```

        strokecolor=:purple,color=( :purple,0.5))

hist!(ax1,ERC_null.fERC,bins=100,
      label="Null Expectation",normalization=:pdf,
      strokecolor=:orange,color=( :orange,0.5))

hist!(ax1,ERC.fERC,bins=200,
      label="Real Data",normalization=:pdf,
      strokecolor=:blue,color=( :blue,0.5))
vlines!(ax1,quantile(ERC_null.fERC,[0.95]),color=:orange)
vlines!(ax1,quantile(ERC_sims.fERC,[0.95]),color=:purple)

xlims!(ax1,(-5,15))

axislegend(ax1,merge=true,unique=true)

ax2, hm = hexbin(fig1[1,2],ERC.n_edges,ERC.fERC,
  colormap=[Makie.to_color(:transparent);Makie.to_colormap(:viridis)],
  axis= (; xgridvisible=false,ygridvisible=false,
    ylabel="fERC",xlabel="Shared Edges"),
  colorscale=Makie.pseudolog10,
  bins=(50,50),
  threshold=2)

#loess_1_2 = loess(collect(ERC.n_edges),collect(ERC.fERC),span=0.1)
#xs = range(5,120,length=200)
#ys = predict(loess_1_2,xs)
#lines!(ax2,xs,ys,linewidth=8,color=:white)
#lines!(ax2,xs,ys,linewidth=4,color=:black)

Colorbar(fig1[1,2][1,2],hm,ticks=[10,100,1000,10000,100000,1000000])

ax3 = Axis(fig1[2,1],yticks=( [0.25,0.5],["",""]),xlabel="fERC",ylabel="Pair on _ chromosomes")

sferc = [ERC.fERC[findall(ERC.same_chr=="same")],ERC.fERC[findall(ERC.same_chr=="different")]]
for i in 1:2
    d = density!(ax3,sferc[i],offset=i./4,color=:x,colormap=:viridis,clorrage=[-5,15],label=i)
    strokecolor=:black,strokewidth=1)
    translate!(d,0,0,-0.1i)
end

```

```

end
xlims!(ax3,(-5,15))
annotation!(ax3,[(1,1/4),(1,1/2)],text=["Same","Different"],
    textcolor=:white,
    fontsize=20)

sub_ERC = findall(!ismissing).(ERC.dist))

ax4,hm = hexbin(fig1[2,2],ERC.dist[sub_ERC],ERC.fERC[sub_ERC],
    colorscale=Makie.pseudolog10,
    bins=(50,50),
    axis=(; xlabel="Pair Distance (bp)",
        ylabel="fERC Score",
        xscale=Makie.pseudolog10,
        xticks=(0,10,1000,100000,10000000),[L"0",L"10",L"10^3",L"10^5",L"10^7"]),
    xgridvisible=false,ygridvisible=false))
Colorbar(fig1[2,2][1,2],hm,ticks=[10,100,1000,10000,100000,1000000])

lm_1 = lm(@formula(fERC~dist),ERC[sub_ERC,:])
xs = range(extrema(ERC.dist[sub_ERC])...,length=200)
ys = predict(lm_1,DataFrame(dist=collect(xs)))
lines!(ax4,xs,ys,linewidth=8,color=:white)
lines!(ax4,xs,ys,linewidth=4,color=:black)

for (label,layout) in zip(["A","B","C","D"],[fig1[1,1],fig1[1,2],fig1[2,1],fig1[2,2]])
    Label(layout[1,1,TopLeft()],label,
        fontsize=26,
        font=:bold,
        padding=(0,5,5,0),
        halign=:right)
end

save(plot_prefix*"fig1.pdf",fig1)

```

Chromosome plot

Julia has no good backend for this (well, neither does R), but I want to plot each pair as a point in a heatmap, with the center of each gene. The issue is converting the location to something meaningful across chromosomes.

Need to have the order of genes consistent between x and y, while accounting for different chromosomes. We'll make dummy X and Y coords here, with the order: 2L,2R,3L,3R,4,X,Y,MT

```
#Pre-compute a bunch of locations/bins
pos_i = (ERC.pos_i_1 .+ ERC.pos_i_2)./2
pos_j = (ERC.pos_j_1 .+ ERC.pos_j_2)./2

chr_order = ["2L","2R","3L","3R","4","X","Y","MT"]
chr_lengths = [23312347,25254477,28012551,32067996,1271129,23107330,3666928]
chr_lines = [sum(chr_lengths[1:x]) for x in 1:7]
chr_label_pos = vcat(chr_lengths[1]/2,[chr_lengths[i]/2+chr_lines[i-1] for i in 2:7])
chr_ticks = (chr_label_pos,chr_order[1:7])

pos_i = chr_order_pos(pos_i,ERC.chr_i,chr_order,chr_lengths)
pos_j = chr_order_pos(pos_j,ERC.chr_j,chr_order,chr_lengths)

#Only plot triangle
pos_map1= [minimum([pos_i[x],pos_j[x]]) for x in 1:length(pos_i)]
pos_map2= [maximum([pos_i[x],pos_j[x]]) for x in 1:length(pos_i)]

#We don't actually want to plot 80 million points, instead we summarize into bins
wnd_size=100000

binned_ERC = binned_mean(DataFrame(p1=pos_map1,p2=pos_map2,r=ERC.r),wnd_size,wnd_size)

#Now locations of significant interactions
pos_i = (sig_ERC.pos_i_1 .+ sig_ERC.pos_i_2)./2
pos_j = (sig_ERC.pos_j_1 .+ sig_ERC.pos_j_2)./2

pos_i = chr_order_pos(pos_i,sig_ERC.chr_i,chr_order,chr_lengths)
pos_j = chr_order_pos(pos_j,sig_ERC.chr_j,chr_order,chr_lengths)

#Only plot lower triangle
pos_map2= [minimum([pos_i[x],pos_j[x]]) for x in 1:length(pos_i)]
pos_map1= [maximum([pos_i[x],pos_j[x]]) for x in 1:length(pos_i)]

gene_pos = chr_order_pos((gene_table.pos_1.+gene_table.pos_2)./2,gene_table.chr,chr_order,chr_order)

# And the actual plotting
```

```

fig2 = Figure()

#Mean gene fERC plot
ax1 = Axis(fig2[1,1],xlabel="fERC",ylabel="Position",
           yticks=chr_ticks,
           yticksiz=0,
           yminorticks=chr_lines,
           yminorticksvisible=true,
           yminorgridvisible=true,
           yminorgridcolor=RGBAf(0,0,0,0.25),
           ygridcolor=RGBAf(0,0,0,0),
           limits=(nothing,nothing,0,maximum(chr_lines)))
scatter!(ax1,gene_table.mean_fERC,gene_pos,color=gene_table.sig_ints,markersize=3)
loess1 = loess(gene_pos,gene_table.mean_fERC,span=0.1)
xs=range(extrema(gene_pos)...;step=100000)
ys = predict(loess1,xs)
lines!(ax1,ys,xs,linewidth=8,color=:white)
lines!(ax1,ys,xs,linewidth=4,color=:black)

#Genome-wide interaction plot
ax2 = Axis(fig2[1,2],xlabel="",ylabel="",
           yticklabelsvisible=false,
           xticklabelsvisible=false,
           xminorticks=chr_lines,
           yminorticks=chr_lines,
           xticksiz=0,
           yticksiz=0,
           xminorticksvisible=true,
           yminorticksvisible=true,
           yminorgridvisible=true,
           yminorgridcolor=RGBAf(0,0,0,0.25),
           ygridcolor=RGBAf(0,0,0,0),
           xminorgridvisible=true,
           xminorgridcolor=RGBAf(0,0,0,0.25),
           xgridcolor=RGBAf(0,0,0,0)
)
heatmap!(ax2,binned_ERC.x_bin,binned_ERC.y_bin,binned_ERC.r_mean)
scatter!(ax2,pos_map1,pos_map2,color=:red,markersize=1)

#Significant interaction plot

```

```

ax3 = Axis(fig2[2,2],xlabel="Position",ylabel="Degree",
          xticks=chr_ticks,
          xticks=0,
          xminorticks=chr_lines,
          xminorticksvisible=true,
          xminorgridvisible=true,
          xminorgridcolor=RGBAf(0,0,0,0.25),
          xgridcolor=RGBAf(0,0,0,0),
          yticks=[1,400,800,1200],
          limits=(0,maximum(chr_lines),nothing,nothing))
scatter!(ax3,gene_pos,gene_table.sig_ints,color=gene_table.sig_ints,markersize=2)

#Resize for emphasis on interaction plot
rowsize!(fig2.layout,2,Auto(0.2))
colsize!(fig2.layout,1,Auto(0.2))
colgap!(fig2.layout,-15)
rowgap!(fig2.layout,-15)

save(plot_prefix*"fig2.pdf",fig2)

```

Community detection

We next detect communities of genes based on co-evolutionary interactions. We use the Louvain clustering approach, which tries to maximize modularity within each community (e.g. how dense interactions are within each community versus out of it). We use the Leiden algorithm approach implemented in R. We can try to get R and Julia to interoperate using RCall, but we've found in testing this is unreliable for some users. Please modify the location of your files in the below code.

```

require(igraph)
require(readr)

sig_ERC = read_csv("data/dipterans/D_melanogaster_sig_ints.csv")
edges = c(apply(sig_ERC,1,function(x) c(x[1],x[2])))
graph = make_graph(edges,directed=FALSE)

#Identify only larges connected component
comps = components(graph)
drop = names(comps$membership)[which(comps$membership != 1)]
subgraph = delete_vertices(graph,drop)

```

```
comms = cluster_leiden(subgraph,objective_function="modularity",resolution=1,n_iterations=1000)

comms$membership = match(1:max(comms$membership),order(table(comms$membership)),decreasing=TRUE)

write_tsv(data.frame(name=comms$names,community=comms$membership),file="data/dipterans/D_melanogaster_communities.tsv")
```

Now, some quick code to analyze the gene trees of each community:

```
comms = CSV.read("data/dipterans/D_melanogaster_communities.tsv",DataFrame)

##make Astral consensus trees

## Remove support values so we can read them in

## Read in trees
spectre = read_tree("data/dipterans/D_melanogaster/trees/astal_consensus.newick")
comm_trees = [read_tree("data/dipterans/D_melanogaster/trees/comm_$x.newick") for x in 1:5]
main_table = breakdown_tree(spectre)
tables = [breakdown_tree(comm_trees[x]) for x in 1:5]
for x in 1:5
    rename!(tables[x],:bl=>"scaled_$x")
    main_table = leftjoin(main_table,tables[x],on=:node)
    main_table[:, "scaled_$x"] = main_table[:, "scaled_$x"] ./ main_table.bl
end

CSV.write("data/dipterans/D_melanogaster/trees/rescaled.table",main_table)

#This makes a table where the species tree branches are colored by a scale of how rapidly the
```

And then we plot in R (because ggtree is nice)

```
library(ggtree)
library(readr)
species_tree = read.tree("data/dipterans/D_melanogaster_communities.tre")
```

GO analysis

The resulting communities are examined for GO category enrichment. The easiest approach here is to use `gprofiler2`, which has a nice implementation in R, so once again we'll step out of Julia for a bit, using `RCall` to make the calls from inside julia. You will need R and

gprofiler2 installed for this to work. Alternatively, you can run the analysis in R, but it will require some data transfer

```
require(readr)
require(gprofiler2)
require(ggplot2)
require(GOSeSim)
require(org.Dm.eg.db)
require(clusterProfiler)
require(enrichplot)

gene_table = read_csv("data/dipterans/D_melanogaster_annotated_gene_table.csv")
#Five major clusters
big_comms = which(table(comms$community)>100)
cs = lapply(1:length(big_comms),function(x){
  genes = comms$name[which(comms$community == x)]
  idx = match(genes,gene_table$gene)
  stable_ids = gene_table$stable_id[idx]
  stable_ids = stable_ids[!is.na(stable_ids)]
  stable_egs = mapIds(org.Dm.eg.db,keys=stable_ids,column="ENTREZID",keytype="FLYBASE",multi=TRUE)
  return(stable_egs)
})
names(cs) = sapply(1:length(big_comms),function(x) paste("comm_",x,sep=""))

genes=gsub(".treefile","",list.files("data/dipterans/D_melanogaster/trees",pattern=".treefile",full.names=TRUE))
stable_genes = sapply(genes,function(x) gene_table$stable_id[which(gene_table$gene == x)][1])
stable_EGs = mapIds(org.Dm.eg.db,keys=stable_genes,column="ENTREZID",keytype="FLYBASE",multi=TRUE)

gostres = gost(cs,organism="dmelanogaster",custom_bg=names(stable_EGs),ordered_query=FALSE,highlight=TRUE)
write_csv(gostres$result,file="data/dipterans/D_melanogaster_fullGOProfiler.csv")
#After inspection of results, we chose some top terms to highlight for export tables, each table has 10 terms
highlighted = gostres$result[which(gostres$result$highlighted),]
write_csv(highlighted,file="data/dipterans/D_melanogaster_highlighted.csv")

emmap_cluster = function(gene_list,cats=30) {
  ego = enrichGO(gene=gene_list,universe=as.character(stable_EGs),OrgDb=org.Dm.eg.db,ont="BP")
  edo = pairwise_termsim(ego)
  return(emapplot(edo,showCategory=cats))
}
```

```

#Figure SX
obs = table(sig_ERC$chr_combo)
expo = sapply(names(obs),function(x){
  chrs=strsplit(x,split="-");
  scale=1;
  if(chrs[[1]][[1]]==chrs[[1]][[2]]){
    choose(length(which(gene_table$chr==chrs[[1]][[1]])),2)
  } else {
    length(which(gene_table$chr==chrs[[1]][1]))*length(which(gene_table$chr==chrs[[1]][2]))
  }
})
perc = length(sig_ERC$r)/(choose(length(gene_table$gene),2))
expo = expo * perc
rat = obs/expo
dev_table = tibble(obs=as.numeric(obs),exp=floor(as.numeric(expo)),ratio=as.numeric(rat))
chrs = strsplit(names(obs),split="-")
dev_table$chr_i = unlist(lapply(chrs,function(x) x[1]))
dev_table$chr_j = unlist(lapply(chrs,function(x) x[2]))

require(ggplot2)
require(cowplot)
theme_set(theme_cowplot())

chr_plot = ggplot(dev_table,aes(x=chr_i,y=chr_j))+geom_tile(aes(fill=ratio))+
  scale_fill_continuous(type="viridis",trans="log2",breaks=c(0.25,0.5,1,2,4,10),labels=c("0.25","0.5","1","2","4","10"))+
  geom_text(aes(label=obs),vjust=-1,color="white")+
  geom_text(aes(label=paste("(",exp,")",sep="")),vjust=1,color="white")+
  labs(x="Chr 1",y="Chr 2",fill="O(Interactions)/E(Interactions)")

ggsave(chr_plot,file="data/dipterans/plots/D_melanogaster_chr_enrichment.pdf",width=12,height=12)

```

Expression analysis

Stricter GO enrichment/annotation of unknown genes

Two problems exist with our current GO analysis when it comes to individual gene enrichment. While it's true that a community might be enriched for, say, transportation genes in general, when examining individual genes, we have to account for the structure of the network. Suppose transportation genes *tend* to have many significant edges/high fERC scores. Then, for any gene, we expect its GO results to include transportation genes. Highlighting such categories is interesting in and of itself, however, it's worth trying to ask whether for any particular gene, it

co-evolves with any GO set *more* than you would expect given how much co-evolution with that set there tends to be. To perform this analysis, we need to first create a null-expectation. Here, we loop over every GO category, find all members of that category, and extract their average fERC values. Then, for each gene and each GO category, we can ask whether the co-evolutionary signal (in terms of fERC) for that category is greater for that gene than you would expect given random chance. We do this by running a Kolmogorov Smirnov test to check if the observed fERC distribution is different than the expected, and then running a t-test to check if the fERC values are greater than expected. (although, genes with less co-evolution with some categories than expected might be interesting in their own right). The functions to run these analyses are in the `Annotation_functions.jl` script in the `src` folder. The GO annotations are downloaded from FlyBase, while the `gene_table` object created earlier in the script is used to gauge average fERC values expected.

```
include(scrdir("Annotation_functions.jl"))
gene_table=CSV.read(data_dir*focal*"_annotated_gene_table.csv",DataFrame)
go_cats = open_GO(data_dir*"gene_anno.gaf.gz")

expected_GO = go_null(go_cats)
#add descriptions to go terms
annotate_GO_table!(expected_GO,"data/dipterans/go.obo")
#And now generate per-gene results
for g in gene_table.gene
    res = ERC_GO_extend(g,ERC,gene_table,go_cats,expected_GO)
    res = subset(res,:pval_BH => p-> p .< 0.05)
    if !ismissing(res.pval)
        res.diff = res.obs_fERC .- res.exp_fERC
        sort!(res,:diff)
        CSV.write("data/dipterans/D_melanogaster/gene_enrichment/"*g*".csv",res)
    end
end
end
```

Specificity figure

```
id_i = indexin(ERC.i,gene_table.gene)
id_j = indexin(ERC.j,gene_table.gene)

cor_fERC = 2 .* ERC.fERC .- gene_table.mean_fERC[id_i]-gene_table.mean_fERC[id_j]

points = [Point3f(pos_map1[x],pos_map2[x],cor_fERC[x]) for x in 1:length(pos_map1)]
```

```

fig = Figure()
#Genome-wide interaction plot
ax2 = Axis(fig[2,1],
    xminorticks=chr_lines,
    yminorticks=chr_lines,
    xticksize=0,
    yticksize=0,
    xminorticksvisible=true,
    yminorticksvisible=true,
    yminorgridvisible=true,
    yminorgridcolor=RGBAf(0,0,0,0.25),
    ygridcolor=RGBAf(0,0,0,0),
    xminorgridvisible=true,
    xminorgridcolor=RGBAf(0,0,0,0.25),
    xgridcolor=RGBAf(0,0,0,0),
    xlabel="Position",
    ylabel="Position",
    xticks=chr_ticks,
    yticks=chr_ticks
)
ds = datashader!(ax2,points,agg=Makie.AggMean(),operation=identity,colormap=reverse(diverging,
Colorbar(fig[1,1],ds,vertical=false,label=L"2fERC_{i,j}-\overline{fERC_{i}}}-\overline{fERC_{j}}",
rowgap!(fig.layout,0)

```